

Final Architecture Selection

ELEC 5803

Jesse Levine (101185127)

1 Project Evolution

This project evolved through five reports that directly informed the final architecture. Report 4 is intentionally omitted here because it did not influence the final design path.

Report 1: Baseline RV32I Build

Report 1 established the starting point: a reproduced HLS RV32I processor based on a unified memory architecture. The main contribution of that phase was methodological rather than architectural. It demonstrated how bare-metal RISC-V programs could be compiled into hexadecimal memory images, loaded into the HLS model, and evaluated through C/RTL co-simulation. That phase created the baseline measurement framework for clock frequency, cycle count, and functional correctness.

Report 2: Domain Selection and RV32IM Extension

Report 2 selected machine-learning inference acceleration as the project domain, with SoftMax chosen as the target kernel because of its expensive exponentiation, accumulation, and normalization stages. To make that domain practical on the CPU, the core was extended from RV32I to RV32IM through the introduction of multiplication support. This phase therefore moved the project from “general processor reproduction” to “domain-capable processor baseline”.

Report 3: SoftMax Accelerator Proposal

Report 3 proposed the specialization strategy that ultimately became the final architecture. The key design decisions were already visible at this point: a memory-mapped accelerator, shared memory between processor and accelerator, approximate exponentiation through LUT/PLA methods, and a pipelined hardware path for reduction and normalization. This report defined the architectural thesis that the rest of the project validated.

Report 5: Pipelined Core and Initial Accelerator

Report 5 delivered the first meaningful performance comparison. The RV32IM core was pipelined, and a first standalone SoftMax accelerator was implemented and benchmarked. That report showed two critical results: the software SoftMax running on the core scaled linearly but with a very high per-element cost, and the dedicated accelerator reduced latency dramatically while also preserving the SoftMax sum much more accurately as N increased.

Report 6: Final Accelerator and SoC Integration

Report 6 transformed the standalone accelerator into a complete system by integrating it with the processor as a memory-mapped peripheral inside the Prometheus_SoC. The architecture used shared BRAM, a Zynq-based control plane, and MMIO control registers so the CPU could configure the accelerator and then read results back from the same memory space. That report established the final design direction: the selected architecture was no longer “core only” or “accelerator only,” but a complete core-plus-accelerator SoC.

2 Final Architecture Selection

The selected final architecture is the integrated Prometheus_SoC: a RISC-V core combined with a dedicated SoftMax accelerator, implemented as a shared-memory, MMIO-controlled subsystem and deployed on the PYNQ-Z1.

The architectural decision is justified by two observations. First, the software-only core is instruction-bound; the entire SoftMax computation must be emulated through sequential fixed point arithmetic, causing latency to scale with a large per-element cost and causing the normalization sum to degrade significantly as N grows. Second, the integrated SoC offloads the expensive numerical work into specialized hardware that exploits pipelining, SIMD-style memory access, and dedicated arithmetic structures. This reduces the per-element cost to less than one clock cycle in the final integrated measurements and preserves the SoftMax sum near unity across the full tested range.

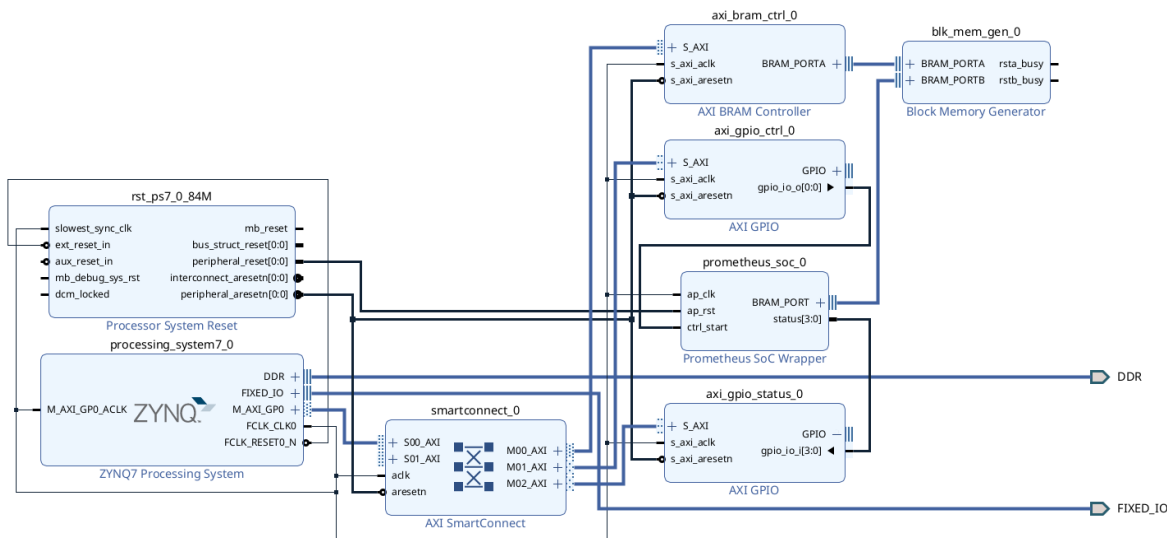


Figure 1: Vivado block diagram of the implemented Prometheus_SoC on the PYNQ-Z1, reused from Report 6. The figure shows the Zynq processing system, AXI SmartConnect, AXI BRAM controller, GPIO control/status path, shared BRAM, and the packaged Prometheus SoC wrapper.

Figure 1 shows the actual Vivado block design used for deployment on the PYNQ-Z1. The Zynq processing system acts as the host-side controller and provides the AXI master interface, clocking, and reset distribution for the programmable logic. Through the AXI SmartConnect, the processing system can access the BRAM controller and the GPIO peripherals that expose the SoC control and status path. The shared BRAM is the central storage element of the design: it holds the RISC-V program image, the input logits, the output probability vector, and the debug values read back after execution.

The packaged Prometheus_SoC wrapper contains the RISC-V core together with the integrated SoftMax accelerator. From the software point of view, the accelerator is invoked through memory-mapped registers, so the CPU can write the input base address, output base address, debug base address, and vector length before asserting the start signal. From the hardware point of view, both the core and the accelerator operate over the same BRAM address space, which avoids explicit data copying or DMA movement. This is the key systems insight of the final architecture: the CPU remains responsible for orchestration and control flow, while the accelerator is tightly integrated

into the same memory system and handles the computationally intensive SoftMax computation directly.

3 Core-Only vs. Core + Accelerator SoC

The core-versus-SoC comparison is based on the implementation-level datasets developed in the earlier reports. This is the correct comparison space for architecture selection because it compares the software-only baseline against the final integrated architecture without introducing the separate timing-closure issue of the standalone FPGA baseline wrapper.

3.1 Implementation Summary

Table 1: Implementation-level resource summary used for architectural comparison. The baseline core values come from the pipelined RV32IM implementation flow, while the SoC values come from the Prometheus_SoC Vivado IP-flow implementation.

Resource / Metric	Core Only	Core + Accelerator SoC
LUT	2169	26090
FF	1465	52339
DSP	12	32
BRAM	0	9
Post-route period (ns)	11.345	11.514

The area increase is expected: the final SoC includes not only the CPU but also the complete SoftMax accelerator datapath, additional buffering, and the memory-mapped control structure needed for hardware-software interaction. The architectural question is therefore whether the added area is justified by the latency and numerical gains. Figures 2–3 show that it is.

3.2 Latency, Numerical Stability, and Speedup

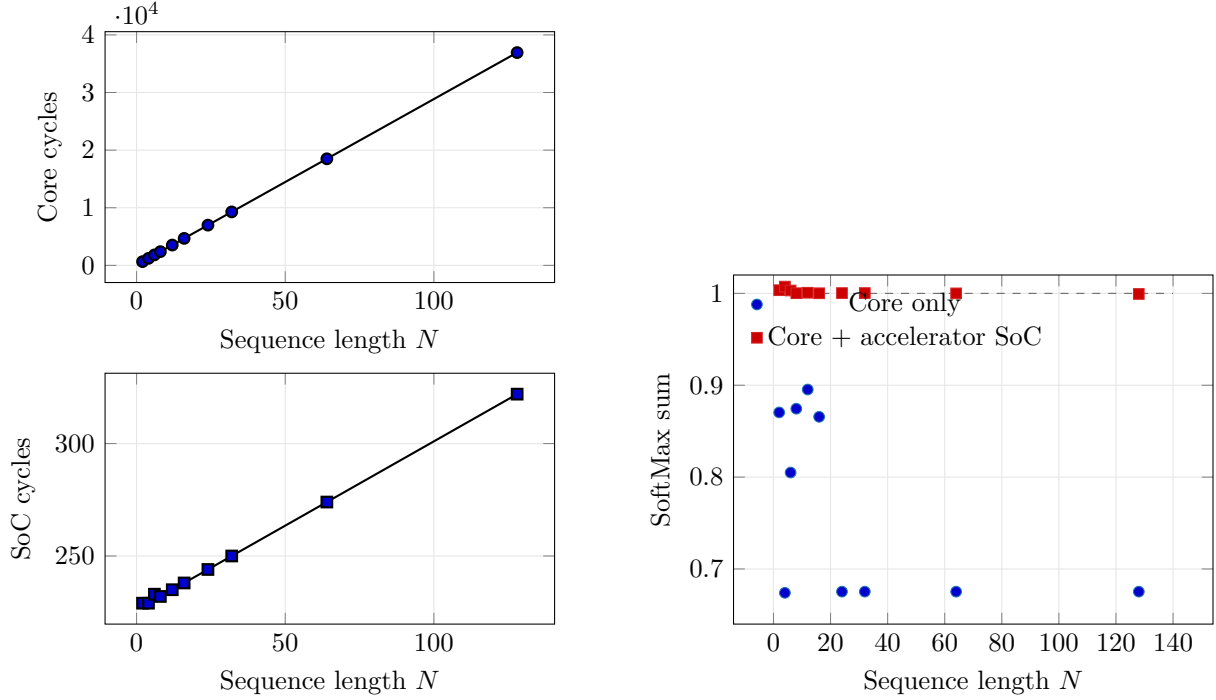
The architectural conclusion is unambiguous. The core-only implementation exhibits a high per-element cost and significant numerical degradation as the vector length increases. In contrast, the integrated SoC keeps the SoftMax sum extremely close to 1.0 and reduces cycle count by more than an order of magnitude even at moderate values of N . By $N = 128$, the measured cycle ratio exceeds $114\times$, while the linear-fit asymptote from the earlier implementation study indicates a theoretical maximum speedup of approximately $389\times$ for sufficiently large N .

4 FPGA Implementation on the PYNQ-Z1

The final project requirement was not only to select an architecture, but to actually deploy the full SoC to a physical FPGA. That requirement was satisfied by implementing Prometheus_SoC on the Digilent PYNQ-Z1, which uses the Xilinx Zynq-7020 (xc7z020c1g400-1) device.

4.1 Implemented PYNQ-Z1 SoC Summary

This final implementation result is significant for two reasons. First, the full system met timing after actual board-level integration, which means the accelerator benefit survives beyond HLS



(a) Latency scaling on separate axes, since a shared axis hides the SoC behavior.

(b) Normalization comparison between the software-only core and the integrated SoC.

Figure 2: Implementation-level comparison between the software SoftMax running on the pipelined core and the final integrated Prometheus_SoC.

co-simulation and beyond isolated IP export. Second, the design was exercised on the physical PYNQ-Z1 through the Zynq processing system, AXI interconnect, shared BRAM, and the GPIO control/status path used for run-time launch and monitoring.

4.2 FPGA Verification Against Simulation

The goal in this section was to show that the *implemented SoC* behaved on the FPGA in the same way as it did in the earlier simulation and implementation studies. Table 3 provides that evidence directly.

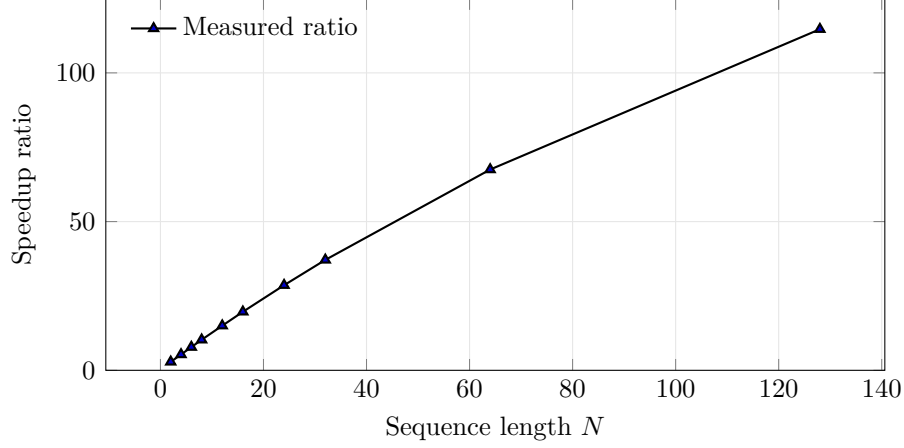
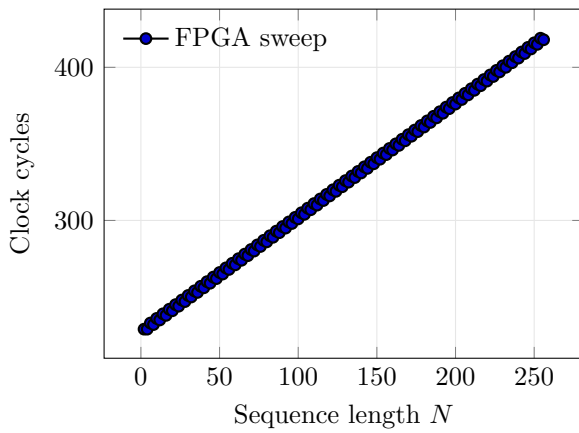


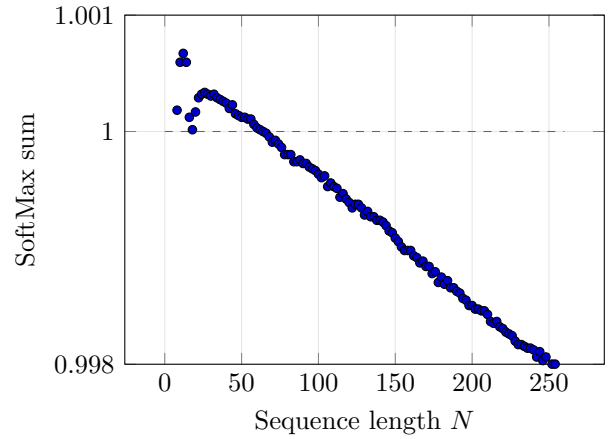
Figure 3: Speedup of the integrated SoC over the software-only core. Based on the linear models from the prior implementation study, the asymptotic maximum speedup is $\frac{287.89}{0.74} \approx 389\times$.

Table 2: Actual PYNQ-Z1 implementation summary for the full Prometheus_SoC. This table refers to the complete board-integrated design, not only the HLS IP block.

Metric	Implemented SoC on PYNQ-Z1
Board	Digilent PYNQ-Z1
FPGA	xc7z020c1g400-1
Clock period (ns)	11.875
Clock frequency (MHz)	84.211
WNS (ns)	0.332
WHS (ns)	0.010
Slice LUTs	32717
Slice Registers	60358
Block RAM Tiles	68.5
DSPs	32
Timing status	Met



(a) Latency



(b) FPGA normalization behavior

Figure 4: Hardware behavior of the implemented Prometheus_SoC on the PYNQ-Z1 across the full FPGA sweep.

Table 3: Selected simulation-versus-FPGA verification points for the implemented SoC.

N	Sim cycles	FPGA cycles	Sim sum	FPGA sum	FPGA latency (μ s)
2	229	229	1.003448	1.003448	2.719
8	232	232	1.000183	1.000183	2.755
32	250	250	1.000320	1.000320	2.969
64	274	274	1.000000	1.000000	3.254
128	322	322	0.999344	0.999344	3.824

The FPGA data confirms that the implemented design reproduces the expected system behavior. Across the complete hardware sweep from $N = 2$ to $N = 256$, the measured SoC latency remained low and scaled gently with vector length, while the SoftMax sum remained near unity. Representative FPGA measurements include 229 cycles at $N = 2$, 274 cycles at $N = 64$, 322 cycles at $N = 128$, and 418 cycles at $N = 256$. At the largest tested vector size, the probability sum remained 0.997955, which is entirely consistent with fixed-point quantization rather than architectural error. Table 3 shows that the FPGA measurements matched the earlier SoC simulation values almost exactly at the sampled points.

This is the key verification result of the project: the selected architecture was not only attractive in simulation, but also functionally realized on the target FPGA and observed to behave almost exactly as predicted by the earlier studies.

5 Final Selection

The final architecture selection is therefore the integrated Prometheus_SoC. It is the only design in the project that satisfies all of the important objectives simultaneously:

- it preserves the RISC-V core as the system controller rather than replacing the processor entirely,
- it removes the SoftMax bottleneck through dedicated hardware,
- it maintains numerical stability much more effectively than the software-only core,
- it meets timing in the final board-level implementation, and
- it has been exercised successfully on the actual PYNQ-Z1 FPGA.

The software-only core remains important as the historical baseline because it quantifies the bottleneck that motivated the accelerator. However, the final deliverable is not the baseline core. The final deliverable is the verified SoC implementation on FPGA, and the results show that this was the correct architectural end point for the project.

6 AI Usage

Codex CLI and ChatGPT were used throughout the final stage of the project. In this deliverable, AI assistance was used to organize the prior report evidence, recover numerical datasets from stored spreadsheets and implementation reports, generate the LaTeX report structure, and prepare native $\LaTeX$ plots and tables. AI was also used during the FPGA deployment stage to help construct the JTAG execution flow and validation scripts for the PYNQ-Z1 hardware.